CATEGORY C

Data Structure Practice Problems

Complete Answer Key

STEP SEM-3 · CodInClub | Sorting & Data Structure Practice

Contents

Part A — 10 coding problems with Java solutions, explanations, sample results, and complexity analysis.

Part B — 10 multiple-choice questions with answers and brief explanations.

Part C — 5 conceptual questions with reasoned answers.

Solutions are written in Java and use standard library classes where appropriate.

Part A — Coding Problems

The implementations below are method-level Java solutions. Add the stated imports at the top of the Java file. The ISBN values are kept as strings so leading zeroes are preserved.

Common imports: `java.util.*;`

Problem 1: Library Catalog Lookup

Approach: Because the catalog is sorted by ISBN, use binary search. Compare the target with the middle ISBN and discard half of the remaining range after each comparison.

Java solution:

```
static class Book {
    String isbn;
    String title;

    Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }
}

static String findBook(List<Book> catalog, String targetIsbn) {
    int low = 0, high = catalog.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        int cmp = catalog.get(mid).isbn.compareTo(targetIsbn);

        if (cmp == 0) return catalog.get(mid).title;
        if (cmp < 0) low = mid + 1;
        else high = mid - 1;
    }
    return "Not Found";
}
```

Sample result: For target `"0003334445"`: **Classic Mythology**. For `"0009998887"`: **Not Found**.

Complexity: Time: **O(log n)** per query. Extra space: **O(1)**. Binary search is appropriate because the data is already sorted and queries greatly outnumber updates; occasional updates can be handled separately while each lookup remains fast.

Problem 2: Warehouse Bin Grid Scan

Approach: Visit every cell once. Maintain a running sum and the largest value seen with its row and column. Update the maximum only when a strictly larger value is found; this preserves the first coordinate when there is a tie.

Java solution:

```
static class WarehouseResult {
    long total;
    int maxRow, maxCol;

    WarehouseResult(long total, int maxRow, int maxCol) {
        this.total = total;
        this.maxRow = maxRow;
        this.maxCol = maxCol;
    }
}

static WarehouseResult warehouseSummary(int[][] grid) {
    long total = 0;
    int maxValue = -1, maxRow = -1, maxCol = -1;

    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[r].length; c++) {
            int value = grid[r][c];
            total += value;

            if (value > maxValue) {
                maxValue = value;
                maxRow = r;
                maxCol = c;
            }
        }
    }
    return new WarehouseResult(total, maxRow, maxCol);
}
```

Sample result: For `[[4,9,2],[7,1,6],[3,12,5]]`: total = **49**, maximum coordinate = **(2, 1)** (zero-based indexing).

Complexity: Time: **O(m × n)**, because every cell is visited once. Extra space: **O(1)**, excluding the input grid.

Problem 3: Transaction Duplicate Check

Approach: Use a HashSet. Before adding each ID, check whether it is already present. A repeated ID immediately proves that a duplicate exists.

Java solution:

```
static boolean hasDuplicate(List<String> transactionIds) {
    Set<String> seen = new HashSet<>();

    for (String id : transactionIds) {
        if (!seen.add(id)) {
            return true;
        }
    }
    return false;
}
```

Sample result: For `["TXN1001","TXN1002","TXN1003","TXN1002","TXN1004"]`: **true**. For `["TXN2001","TXN2002","TXN2003"]`: **false**.

Complexity: Average time: **O(n)**. Extra space: **O(n)** in the worst case. A lower-memory alternative is to sort the IDs and compare adjacent elements: O(n log n) time and typically O(1) or O(log n) auxiliary space depending on the sorting method. The HashSet trades memory for faster average runtime.

Problem 4: Employee ID Lookup in an Unsorted Directory

Approach: Build a HashMap once, mapping each employee ID to the department. Repeated lookups then use the key rather than scanning the original list.

Java solution:

```
static Map<String, String> buildLookup(
        List<String[]> records) {
    Map<String, String> lookup = new HashMap<>();

    for (String[] record : records) {
        lookup.put(record[0], record[1]);
    }
    return lookup;
}

static String getDepartment(
        Map<String, String> lookup, String employeeId) {
    return lookup.getOrDefault(employeeId, "Unknown");
}
```

Sample result: For employee ID `"E390"`: **Marketing**. For `"E999"`: **Unknown**.

Complexity: Building the map: average **O(n)** time. Each lookup: average **O(1)** time. Space: **O(n)**. This is suitable when lookups are frequent and updates are rare.

Problem 5: Pair With Target Sum (Unsorted)

Approach: Use a HashSet for a one-pass solution. For each number x, check whether target – x has already appeared. Check before inserting x so the same array element is not used twice.

Java solution:

```
static boolean hasPairWithSum(int[] nums, int target) {
    Set<Integer> seen = new HashSet<>();

    for (int x : nums) {
        int needed = target - x;
        if (seen.contains(needed)) return true;
        seen.add(x);
    }
    return false;
}
```

Sample result: For [2, 7, 11, 15], target = 9: **true** (2 + 7). For [3, 4, 6], target = 20: **false**.

Complexity: HashSet approach: average **O(n)** time and **O(n)** extra space. Brute force: **O(n²)** time and **O(1)** extra space. Another option is sorting then using two pointers: **O(n log n)** time; sorting may require additional memory depending on the sort.

Problem 6: Best Single Trade Profit

Approach: Scan the prices once. Keep the minimum price seen so far and calculate the profit from selling at each current price. Update the best profit whenever a larger profit is found.

Java solution:

```
static int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
    int bestProfit = 0;

    for (int price : prices) {
        if (price < minPrice) {
            minPrice = price;
        } else {
            bestProfit = Math.max(bestProfit, price - minPrice);
        }
    }
    return bestProfit;
}
```

Sample result: For [7, 1, 5, 3, 6, 4]: **5** (buy at 1, sell at 6). For [7, 6, 4, 3, 1]: **0**.

Complexity: Single scan: **O(n)** time and **O(1)** space. Checking every buy/sell pair: **O(n²)** time and **O(1)** space. The single-pass method scales better because it updates the minimum and best profit as it goes.

Problem 7: Insert Position in a Sorted List

Approach: Use binary search to find the lower-bound position: the first index whose value is greater than or equal to the target. If the target is present, this is its index; otherwise it is the correct insertion index.

Java solution:

```
static int searchInsertPosition(int[] nums, int target) {
    int low = 0, high = nums.length;

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (nums[mid] < target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return low;
}
```

Sample result: For [1, 3, 5, 6], target = 5: **2**. For target = 2: **1**.

Complexity: Time: **O(log n)**. Extra space: **O(1)**. Binary search is appropriate for frequent lookups in a large sorted array.

Problem 8: Spiral Order Traversal

Approach: Maintain top, bottom, left, and right boundaries. Traverse the top row, right column, bottom row, and left column in clockwise order, shrinking the relevant boundary after each traversal. Boundary checks prevent revisiting cells in single-row or single-column layers.

Java solution:

```
static List<Integer> spiralOrder(int[][] grid) {
    List<Integer> result = new ArrayList<>();
    if (grid == null || grid.length == 0 ||
            grid[0].length == 0) return result;

    int top = 0, bottom = grid.length - 1;
    int left = 0, right = grid[0].length - 1;

    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; c++)
            result.add(grid[top][c]);
        top++;

        for (int r = top; r <= bottom; r++)
            result.add(grid[r][right]);
        right--;

        if (top <= bottom) {
            for (int c = right; c >= left; c--)
                result.add(grid[bottom][c]);
            bottom--;
```

```
        }

        if (left <= right) {
            for (int r = bottom; r >= top; r--)
                result.add(grid[r][left]);
            left++;
        }
    }
    return result;
}
```

Sample result: For [[1,2,3],[4,5,6],[7,8,9]]: **[1, 2, 3, 6, 9, 8, 7, 4, 5]**.

Complexity: Time: **O(m × n)**, because each cell is added exactly once. Output space: **O(m × n)**.

Problem 9: Pair Sum in a Sorted Array

Approach: Use two pointers, one at each end. If their sum is too small, move the left pointer right; if too large, move the right pointer left. Since the array is sorted, each move safely eliminates impossible pairs.

Java solution:

```
static int[] pairSumSorted(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left < right) {
        int sum = nums[left] + nums[right];

        if (sum == target) {
            return new int[] { nums[left], nums[right] };
        } else if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return null; // no pair exists
}
```

Sample result: For [-4, -1, 0, 3, 5, 9], target = 4: **(-1, 5)**. For [1, 2, 3], target = 100: **Not Found** (null).

Complexity: Two pointers: **O(n)** time and **O(1)** auxiliary space. A brute-force solution that ignores sorted order takes **O(n²)** time and O(1) space.

Problem 10: Remove Duplicates From a Sorted Array In-Place

Approach: Use a slow pointer to mark the position of the next unique value and a fast pointer to scan the array. When a new value is found, copy it to the next unique position. The first k elements then contain the unique values.

Java solution:

```
static int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;

    int unique = 1;
    for (int scan = 1; scan < nums.length; scan++) {
        if (nums[scan] != nums[unique - 1]) {
            nums[unique] = nums[scan];
            unique++;
        }
    }
    return unique;
}
```

Sample result: For [1, 1, 2, 2, 2, 3, 4, 4, 5]: returned length = **5**; first five elements = **[1, 2, 3, 4, 5]**.

Complexity: Time: **O(n)**. Extra space: **O(1)**. It is in-place because it rearranges values within the original array and does not create another array proportional to the input size.

Part B — Quiz

Choose the best answer. Questions marked “Select all that apply” may have more than one correct option.

Q1. Primary purpose of a data structure

Correct answer: B

A data structure organizes, stores, and provides access to data efficiently for particular operations.

Q2. Description of a non-primitive data structure

Correct answer: D

It is formed by combining primitive types and can store a collection of values.

Q3. Accessing a 1D array element at a known index

Correct answer: C — O(1)

Array indexing computes the element location directly, so access takes constant time.

Q4. Two nested loops, each iterating n times

Correct answer: C — $O(n^2)$

The inner loop runs n times for each of n outer-loop iterations: n × n.

Q5. Separate independent loops of m and n iterations

Correct answer: B — O(m + n)

The work is added, not multiplied, because the loops execute one after the other.

Q6. Common data-structure operations (select all)

Correct answer: A, C, D — Deletion, Traversal, Updating

Debugging and compilation are software-development activities, not standard data-structure operations.

Q7. Sorted transaction dates with frequent date-range queries

Correct answer: C — O(log n) search for date ranges

Ordering the array enables efficient binary-search-based location of range boundaries.

Q8. Frequent searches, rare inserts/deletes, and order must be maintained

Correct answer: D — Sorted Array

A sorted array supports logarithmic search and keeps elements ordered; insertions/deletions can be costly.

Q9. Most effective setting for two pointers

Correct answer: C

The technique benefits from sorted data or another exploitable ordering property.

Q10. Primary benefit of sliding window

Correct answer: C

It reuses information from overlapping windows and can avoid repeated calculations.

Part C — Concept Questions

Question 1: Primitive vs non-primitive data structures

A primitive data type is a basic type directly supported by a programming language and represents a single value, such as an integer, character, or boolean. A non-primitive data structure is built using primitive values and organizes one or more values into a collection or relationship. For example, a Java `enum` value such as `Day.MONDAY` is a simple language-level value, while a graph represented by adjacency lists is a non-primitive structure containing connected vertices and edges. The key distinction is the level of organization and the operations supported, not simply the amount of memory used.

Question 2: Why a linked list is linear

A linked list is linear because its elements form a single sequence: each node (except the last) points to the next node, and traversal follows that one ordered path. Linear versus non-linear describes the logical relationship among elements, not whether they occupy consecutive memory addresses. Linked-list nodes may be scattered in memory, but their logical arrangement is still sequential. Trees and graphs are non-linear because a node can have multiple branches or connections.

Question 3: O(m + n) versus O(m × n)

When one loop processes listA of size m and a separate loop then processes listB of size n, the total work is proportional to m + n, so the complexity is O(m + n). If an m-iteration loop is nested inside an n-iteration loop, the inner work is repeated for every outer iteration, giving m × n operations and O(mn) complexity. The rule is: sequential independent sections add their costs, while nested loops multiply their iteration counts. Big-O notation then ignores constant factors and lower-order terms.

Question 4: Choosing memory to reduce execution time

A practical example is an online store that repeatedly looks up product details by product ID. It can build and maintain a hash map from product ID to product record, using additional memory proportional to the number of products. This changes average lookup time from O(n) for scanning a list to O(1) for a hash-map lookup. The trade-off is extra memory and some work to build or update the map in exchange for faster, frequent searches.

Question 5: Hash map versus sorted array for employee records

A hash map is preferable when the main operations are looking up, inserting, or deleting records by employee ID. Typical hash-map lookup, insertion, and deletion are O(1) on average, with O(n) worst cases; it uses O(n) storage and does not maintain sorted key order. A sorted array is preferable when records must remain ordered and searches are frequent: binary search takes O(log n), and indexed access takes O(1). However, inserting or deleting in a sorted array generally takes O(n) because elements may need to shift. Therefore, choose a hash map for fast key-based updates and lookup, and a sorted array when ordered traversal and range queries are more important.

Complexity quick reference

Problem	Main technique	Time	Extra space
1	Binary search	O(log n)	O(1)
2	Grid traversal	O(mn)	O(1)
3	HashSet	O(n) average	O(n)
4	HashMap	O(n) build; O(1) lookup avg.	O(n)
5	HashSet	O(n) average	O(n)
6	Single scan	O(n)	O(1)
7	Binary search	O(log n)	O(1)

Problem	Main technique	Time	Extra space
8	Spiral traversal	O(mn)	O(mn) output
9	Two pointers	O(n)	O(1)
10	Two pointers	O(n)	O(1)

Submission reminder: Compile and test each Java method in your IDE, and adapt method signatures or input/output handling if your instructor requires a particular class structure. The answers above address all 10 coding problems, 10 quiz questions, and 5 concept questions in the supplied Category C practice sheet.

Source basis: CATEGORY-C-DATA-STRUCTURE_PRACTICE_PROBLEMS.pdf. The accompanying sorting-algorithms introduction was used as supporting context where relevant; the solutions follow the exact tasks and examples in the practice-problems document.